

Practice Exercise 12: Stream Authentication — Hash Chains

Context

We are building a system to authenticate video streams. A server hosts a large file F broken into n blocks $B_0, B_1, \dots, B_n$. We want to verify these blocks one by one as they arrive, without waiting for the entire file to download.

To achieve this, we use a **Hash Chain** constructed backwards:

1. The hash of the last block (B_n) is computed.
2. This hash is appended to the second-to-last block (B_{n-1}).
3. The hash of this augmented block is computed and appended to B_{n-2} , and so on.

Let H be a collision-resistant hash function (SHA256). The relationship is defined as:

$$h_i = H(B_i || h_{i+1})$$

Where h_{i+1} is the hash of the *next* part of the chain (represented as 32 binary bytes). The final hash value h_0 (the hash of the first block augmented with h_1) is the **Root Hash**. This h_0 is trusted and distributed securely.

If a browser trusts h_0 , it can verify B_0 and learn the trusted h_1 for the next step.

Your Task

You will:

1. Implement the backward hashing logic to compute the Root Hash (h_0) from a list of data blocks.
2. Verify a specific stream of blocks against a trusted Root Hash.

Required functions

You must implement **two** functions:

```
def compute_root_hash(blocks):
```

```
    """
```

```
    input:  List of bytes (where each element is a block of data, e.g., 1KB)
```

```
    output: hex string of the final hash h0
```

```
    Note: Process the list backwards.
```

```
          The last block has no hash appended before hashing.
```

```
          ALL other blocks have the binary digest (32 bytes) of the previous
```

step appended.
"""

```
def verify_stream(blocks, trusted_h0):  
    """  
    input:  
        blocks:      List of bytes (as received by the client)  
        trusted_h0:  hex string of the trusted root hash  
  
    output:  
        boolean: True if every single block in the chain is valid, False otherwise.  
  
    Logic:  
        1. Verify  $B_0 || h_1$  matches  $trusted\_h0$ .  
        2. Extract  $h_1$  from the data.  
        3. Verify  $B_1 || h_2$  matches  $h_1$ .  
        4. Repeat.  
    """
```

Implementation Idea

For compute_root_hash: Start from the last element of the list.

1. Hash the last block: $current_hash = \text{SHA256}(blocks[-1])$.
2. Move to $blocks[-2]$. Append the $current_hash$ (as raw bytes, not hex) to it.
3. Update $current_hash = \text{SHA256}(blocks[-2] + current_hash_bytes)$.
4. Repeat until index 0.

For verify_stream: The “blocks” received by the client in this exercise will contain the data B_i **and** the appended hash h_{i+1} (except the very last block).

1. Take the first block (which is $B_0 || h_1$).
2. Hash it and compare with $trusted_h0$.
3. If valid, slice off the last 32 bytes of this block; that is your new trusted hash for the next block.

Dataset

Use the hashlib library in Python (`import hashlib`).

Test Case 1: Small Toy Data

Input Blocks (Hex Encoded): Convert these to bytes before processing.

```
# 3 blocks.  
# block[2] is the end.  
# block[0] is the start.
```

```
blocks_hex = [
    "48656c6c6f20",      # "Hello "
    "576f726c64",        # "World"
    "21"                  # "!"
]
```

Expected Root Hash (h_0):

7efaf53af572f7680607a3ac24b441ad3966784e50dda32613405a0847a0433e

Test Case 2: Verification Challenge

Here is a sequence of blocks (already augmented with the next hashes, except the last one).

Stream Data (Hex Encoded):

Note: In a real scenario, these would be 1KB blocks. Here they are short strings for testing.

```
# Stream Data (Data received by the Client)
stream_data_hex = [
    "5365637572697479" +
    e760f249914465825e27d0c6f6110637307eac934fc225749bfe75324df5db2e,
                                # Block 0: Data "Security" + Hash of the next block
    "20697320" +
    c960563f683a14202dc212ae6554753a03c6cd6ab9df8c27ebd818ba848ab9ef,
                                # Block 1: Data " is " + Hash of the next block
    "46756e"                    # Block 2: Data "Fun" (No trailing hash)
]
```

Hint: The ASCII data hidden in these blocks forms the sentence "Security is Fun".

Trusted Root Hash (h_0):

f11afac36abee3f8be106bc53226012250cad05b2d3ee621bb5857ff94d92d65

Deliverables

A .py file that contains:

- Your two functions.
- Code running `compute_root_hash` on Test Case 1 and printing the result.
- Code running `verify_stream` on Test Case 2 and printing "Stream Valid" or "Stream Tampered".
- `Members.txt` of your group.